

Experiment 8

Convolutional Neural Network for Clothing Image Classification on Fashion-MNIST Dataset

1. Dataset Source

- Source Link: <https://www.kaggle.com/datasets/zalando-research/fashionmnist>

2. Dataset Description

- 60,000 training images + 10,000 test images
- Image size: 28×28 grayscale
- 10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot
- Pixel values: 0–255
- Balanced dataset, no missing values

3. Mathematical Formulation of the Algorithms

1. Convolution Operation

$$Z_{i,j}^{(k)} = \sum_m \sum_n X_{i+m,j+n} \cdot W_{m,n}^{(k)} + b^{(k)}$$

- X : Input image
- $W^{(k)}$: Filter (kernel)
- $b^{(k)}$: Bias
- Produces feature maps

2. Activation Function (ReLU)

$$f(x) = \max(0, x)$$

- Introduces non-linearity
- Removes negative values

3. Pooling (Max Pooling)

$$Y_{i,j} = \max_{(m,n) \in R} X_{i+m,j+n}$$

- Reduces spatial dimensions
- Keeps important features

4. Fully Connected Layer

$$z_j = \sum_i w_{ji} x_i + b_j$$

- Flattens feature maps into vector
- Performs classification

5. Softmax Function (Output Layer)

$$P(y = j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

- Converts outputs into probabilities
- Sum of probabilities = 1

6. Loss Function (Cross-Entropy)

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

- Measures difference between actual and predicted labels

7. Weight Update (Gradient Descent)

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

- η : Learning rate
- Updates weights to minimize loss

4. Algorithm Limitations

- Requires large labeled data
- Computationally expensive
- Sensitive to rotation/scale without augmentation
- Black-box model (low interpretability)
- May misclassify similar classes

5. Methodology / Workflow

Step 1: Set up your environment

```
# Cell 1: Install & import everything
!pip install tensorflow matplotlib seaborn --quiet

import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import time

print("TensorFlow version:", tf.__version__)

TensorFlow version: 2.19.0
```

Step 2: Load the Dataset

```
# Cell 2: Load Fashion-MNIST
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

print("Dataset loaded successfully!")
print("Train shape:", x_train.shape, "Labels:", y_train.shape)
print("Test shape:", x_test.shape, "Labels:", y_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>
29515/29515 — 0s 0us/step
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>
26421880/26421880 — 0s 0us/step
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>
5148/5148 — 0s 0us/step
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>
4422102/4422102 — 0s 0us/step
Dataset loaded successfully!
Train shape: (60000, 28, 28) Labels: (60000,)
Test shape: (10000, 28, 28) Labels: (10000,)

Step 3: Data Preprocessing

```
# Cell 3: Preprocess
x_train = x_train / 255.0
x_test = x_test / 255.0

# Add channel dimension for CNN (28,28) → (28,28,1)
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Preprocessing done. New shape:", x_train.shape)
```

Preprocessing done. New shape: (60000, 28, 28, 1)

Step 4: Build the CNN Model

```
# Cell 4: Build CNN
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)  
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Step 5: Compile & Train the Model

```
# Cell 5: Compile and train  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
start_time = time.time()  
history = model.fit(x_train, y_train,  
                   epochs=20,  
                   batch_size=128,  
                   validation_split=0.1,  
                   verbose=1)  
training_time = time.time() - start_time  
print(f"Training completed in {training_time:.2f} seconds")
```

```
Epoch 1/20  
422/422 — 45s 104ms/step - accuracy: 0.7538 - loss: 0.6888 - val_accuracy: 0.8447 - val_loss: 0.4139  
Epoch 2/20  
422/422 — 77s 92ms/step - accuracy: 0.8466 - loss: 0.4293 - val_accuracy: 0.8697 - val_loss: 0.3462  
Epoch 3/20  
422/422 — 37s 88ms/step - accuracy: 0.8672 - loss: 0.3733 - val_accuracy: 0.8828 - val_loss: 0.3078  
Epoch 4/20  
422/422 — 38s 90ms/step - accuracy: 0.8765 - loss: 0.3417 - val_accuracy: 0.8885 - val_loss: 0.2986  
Epoch 5/20  
422/422 — 39s 91ms/step - accuracy: 0.8855 - loss: 0.3191 - val_accuracy: 0.8957 - val_loss: 0.2771  
Epoch 6/20  
422/422 — 38s 90ms/step - accuracy: 0.8916 - loss: 0.2978 - val_accuracy: 0.8990 - val_loss: 0.2704  
Epoch 7/20  
422/422 — 38s 91ms/step - accuracy: 0.8970 - loss: 0.2860 - val_accuracy: 0.9052 - val_loss: 0.2578
```

```
422/422 — 38s 91ms/step - accuracy: 0.8970 - loss: 0.2860 - val_accuracy: 0.9052 - val_loss: 0.2578  
Epoch 8/20  
422/422 — 38s 90ms/step - accuracy: 0.9035 - loss: 0.2692 - val_accuracy: 0.9078 - val_loss: 0.2462  
Epoch 9/20  
422/422 — 38s 90ms/step - accuracy: 0.9054 - loss: 0.2591 - val_accuracy: 0.9048 - val_loss: 0.2539  
Epoch 10/20  
422/422 — 38s 90ms/step - accuracy: 0.9108 - loss: 0.2435 - val_accuracy: 0.9107 - val_loss: 0.2397  
Epoch 11/20  
422/422 — 41s 90ms/step - accuracy: 0.9144 - loss: 0.2351 - val_accuracy: 0.9158 - val_loss: 0.2294  
Epoch 12/20  
422/422 — 41s 90ms/step - accuracy: 0.9171 - loss: 0.2265 - val_accuracy: 0.9220 - val_loss: 0.2255  
Epoch 13/20  
422/422 — 38s 90ms/step - accuracy: 0.9215 - loss: 0.2127 - val_accuracy: 0.9172 - val_loss: 0.2270  
Epoch 14/20  
422/422 — 40s 88ms/step - accuracy: 0.9250 - loss: 0.2051 - val_accuracy: 0.9187 - val_loss: 0.2263  
Epoch 15/20  
422/422 — 38s 90ms/step - accuracy: 0.9282 - loss: 0.1966 - val_accuracy: 0.9193 - val_loss: 0.2264  
Epoch 16/20  
422/422 — 40s 88ms/step - accuracy: 0.9293 - loss: 0.1896 - val_accuracy: 0.9203 - val_loss: 0.2246  
Epoch 17/20  
422/422 — 41s 89ms/step - accuracy: 0.9321 - loss: 0.1819 - val_accuracy: 0.9215 - val_loss: 0.2270  
Epoch 18/20  
422/422 — 39s 93ms/step - accuracy: 0.9336 - loss: 0.1738 - val_accuracy: 0.9203 - val_loss: 0.2272  
Epoch 19/20  
422/422 — 40s 91ms/step - accuracy: 0.9366 - loss: 0.1689 - val_accuracy: 0.9215 - val_loss: 0.2310  
Epoch 20/20  
422/422 — 39s 92ms/step - accuracy: 0.9370 - loss: 0.1665 - val_accuracy: 0.9195 - val_loss: 0.2340  
Training completed in 825.07 seconds
```

Step 6: Evaluate the Model

```
# Cell 6: Evaluation
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc*100:.2f}%")

# Predictions
y_pred = np.argmax(model.predict(x_test), axis=1)

# Classification report
print(classification_report(y_test, y_pred, target_names=[
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]))
```

Test Accuracy: 91.26%

313/313 ————— 2s 7ms/step

	precision	recall	f1-score	support
T-shirt/top	0.86	0.87	0.87	1000
Trouser	0.99	0.98	0.98	1000
Pullover	0.88	0.86	0.87	1000
Dress	0.93	0.89	0.91	1000
Coat	0.83	0.90	0.86	1000
Sandal	0.99	0.96	0.98	1000
Shirt	0.76	0.74	0.75	1000
Sneaker	0.94	0.98	0.96	1000
Bag	0.98	0.98	0.98	1000
Ankle boot	0.97	0.96	0.96	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

6. Performance Analysis

- **Overall Accuracy:** The model achieved **91.26% accuracy**, indicating good performance.
- **Training Behavior:**
Training and validation accuracy increased steadily, showing effective learning and good generalization. Loss decreased over time, with slight flattening indicating convergence.
- **Classification Performance:**
High accuracy for classes like **Trouser, Sandal, Sneaker, Bag, Ankle boot**.
Lower performance for **Shirt, Pullover, and Coat**, with *Shirt* being most misclassified.
- **Confusion Matrix Insight:**
Errors mainly occur between **visually similar classes** (e.g., Shirt vs T-shirt/top, Pullover, Coat).
Very few errors between clearly different categories.

7. Hyperparameter Tuning

Hyperparameter tuning is used to find the best set of parameters to improve model performance. Instead of manual selection, **Keras Tuner (Random Search)** is used to automatically explore different combinations.

Tuned Parameters

- Number of filters (Conv layers)
- Dense layer units
- Dropout rate
- Learning rate

Impact of Hyperparameter Tuning

- **Improved Accuracy:** Tuned model performs equal or better than baseline
- **Optimal Parameters Found:** Best values for filters, units, dropout, and learning rate
- **Better Generalization:** Reduces overfitting and underfitting
- **High Computational Cost:** Requires training multiple models